Gustavo Garay

Dr. Satzhan Sitmukhambetov

CSCI 3334-02

May 6th, 2026

A Comparison of Two Task Dispatch Policies

Task scheduling is one of the foundational problems in concurrent systems. Whenever multiple workloads share a finite resource such as processing time and memory, there exists an overarching process that determines how fast the system finishes its work and how fairly that work is distributed. This report attempts to compare two task dispatch policies implemented in Rust over a shared worker pool with a bounded CPU budget. The first policy dispatches tasks in a First-In-First-Out (FIFO) manner. The policy dispatches whatever happens to be at the front of the queue that holds incoming tasks. The second policy is optimized to be a budget-aware dispatcher with aging-based starvation prevention. Rather than dispatching the front of the queue unconditionally, this policy inspects the full queue at each decision point and selects the task that best fills the available CPU budget. Each policy's implementations are described, tested, and further stress tested against burst task delivery.

The FIFO implementation dispatches tasks in strict order of arrival. Each worker, when it acquires the queue lock, examines only the front of the queue: if the front task fits the current CPU budget and a worker slot is available, the worker takes it; otherwise, it waits. The implementation uses four thread types working together: a main thread, a queue thread, a set of worker threads, and a monitor thread. The main thread sets up default configurations and shared state, drives a generator loop that spawns tasks into an asynchronous mpsc channel, and joins all

spawned threads at the end before printing the final results. The queue thread receives tasks from the channel and pushes them into a shared VecDeque; it also carries a flag that signals to workers when no more tasks are coming. A configurable number of worker threads (eight by default) each run the same loop: pull a task from the front of the queue, sleep to simulate work, then update the shared state and notify other workers of their completion. The monitor thread wakes every 10 ms, snapshots the current CPU load and active worker count, and appends the snapshot to a vector for end-of-run statistics.

Several pieces of state are shared across these threads. The central one is a QueueState struct holding the task queue, the current active_workers count, the cumulative active_cpu_load, and a generation_done flag. All four fields are bundled in a single Mutex<QueueState> paired with a Condvar, wrapped together in Arc<(Mutex<QueueState>, Condvar)>. Any thread that interacts with the queue, load, or worker count must acquire this mutex. The condvar plays a critical role: when a worker cannot dispatch (because no task fits the current budget, or the queue is empty), it releases the lock and sleeps on the condvar; when another worker finishes a task and frees CPU budget, it broadcasts on the condvar to wake the sleepers so they can re-evaluate.

Three other shared structures support the simulation. The completed vector, Arc<Mutex<Vec<Task>>>, collects finished tasks so the main thread can compute statistics at shutdown. The snapshots vector, Arc<Mutex<Vec<Snapshot>>>, is written only by the monitor and read only by main at shutdown. A shutdown flag wrapped in Arc<AtomicBool> tells the monitor when to stop; an atomic suffices here because the access pattern is one writer, one reader, and no compound operations. The overall protection strategy is one mutex per concern: bundling fields that must move together (queue state) under one lock, and isolating independent data (completed tasks, snapshots) under their own. Arc ensures every thread holds a reference to

the same instance, while Mutex and AtomicBool enforce that only one thread mutates the inner data at a time.

A single mpsc::channel<Task> connects the generator (on the main thread) to the queue thread. The channel implements a clean producer-consumer pattern: the generator never needs to look back at what it sent or wait for acknowledgement, and the channel's recv() method handles termination naturally. When the generator drops the sender, the queue thread's for task in rx loop exits, which is the trigger for setting generation_done. This design avoids sentinel values or out-of-band shutdown signals.

The optimized simulation uses the same four-thread architecture as FIFO with the difference in the dispatch logic. The differences lie inside the worker dispatch loop and in two small additions to shared state. Instead of only examining the front of the queue, each worker performs a four-pass scan over the entire queue at every dispatch opportunity, choosing the task that best fits the current CPU budget under the policy's preference rules.

The first addition is a skip_count field on every Task, initialized to zero. The second is an aging_threshold constant in Config, set to 20, which determines how many times a task may be skipped before the dispatcher is forced to run it. Because skip_count lives inside the Task struct, which is inside the VecDeque and thus inside QueueState, it inherits the protection of the same mutex that already guards the queue so no new locks are needed.

The dispatch policy itself is a four-pass scan over the queue, executed every time a worker considers taking a task. The passes are evaluated in order, and the first match wins. The four passes are: an aged-task hard block, a budget-aware CPU preference, an IO fallback, and a last-resort any-fit.

The aged-task hard block checks whether any task in the queue has a skip_count greater than or equal to the aging threshold. If so, the current worker is restricted to dispatching only an aged task. If an aged task exists but does not fit the current budget, the worker waits rather than fall through to the other passes. This is the strict starvation guarantee, once a task has been skipped 20 times, no other task may dispatch on this worker until that aged task runs.

The budget-aware CPU preference pass runs next if no aged tasks are pending. The dispatcher checks the remaining CPU budget and, if at least 35% is free, scans for the first CPU task that fits. Since eight workers all running IO tasks at 10% each cap CPU consumption at 80%, leaving 20% of the budget permanently idle, mixing in CPU tasks closes that gap (six IO plus one CPU equals 95% budget utilization), increasing total throughput per dispatch cycle.

The IO fallback runs when the budget is too tight for a CPU task, or when no CPU task is queued. The dispatcher selects the first IO task that fits the budget. Since IO tasks cost only 10%, they almost always fit. A last-resort any-fit pass handles the edge case where neither preferred kind is available but some other task could still be dispatched.

When a task is finally chosen at queue position i, every task at positions 0..i has its skip_count incremented before the chosen task is removed. This gives the aging mechanism its information. A task that consistently sits near the front but never gets selected, like a CPU task where budget is repeatedly saturated by IO tasks, accumulates skips quickly and is eventually promoted via the aged-task pass.

The worker's wait and wake behavior is also more nuanced than in FIFO. During normal operation, when a worker finishes a task and frees CPU budget, it calls cvar.notify_one() to wake exactly one sleeping worker rather than broadcasting to all eight. This avoids a thundering-herd

pattern where every worker wakes on every state change, fights for the lock, and goes back to sleep having done nothing useful. The exception is at the tail of the simulation: when generation_done is set and the queue is empty, the worker calls notify_all() instead, ensuring every sleeping worker wakes to check the termination condition and exit cleanly.

When comparing initial non-stressed results between the two implementations, each simulation was initially tested against 1000 tasks sent out every 20ms. Both implementations were then stress-tested against a burst delivery of 200ms intervals instead, where the shape of the task arrivals is changed. The arrival_interval_ms configuration is changed to 200ms from 20ms and a burst_size variable of 10 sends tasks back to back before sleeping again. Ten tasks land at the same moment and sit in the queue together while workers process them. A worker who wakes up mid-burst sees a full menu of options.

The following table shows results across the five metrics that best characterize each policy's performance. Makespan is the total wall-clock time from the start of the simulation to the moment the last task finishes. Average wait time is the mean time a task spends in the queue from arrival until a worker begins executing it. Average turnaround time is the mean time from a task's arrival in the queue to its completion. For all three of these, lower is better. Worker utilization is the average number of active workers divided by total workers, measuring how busy the worker pool was on average, higher is better. Fairness gap is the absolute difference between the average wait time of IO tasks and the average wait time of CPU tasks; a small gap means both task types are served equally, while a large gap means one type is preferred at the expense of the other. Average CPU consumption is the mean percentage of the total available CPU budget that is actively being used over the duration of the simulation.

Metric	FIFO	FIFO-Burst	Optimized	Optimized-Burst
Makespan(ms)	39122	39258	38277	36050
Avg wait(ms)	9152.2	9248.4	8925.6	7712.0
Avg turnaround(ms)	9352.4	9448.6	9125.8	7912.1
Worker utilization	64.0%	63.8%	65.4%	69.4%
Fairness gap(ms)	457.9	455.7	288.6	63.0
Avg CPU consumption	89.5%	89.2%	91.5%	96.6%

The optimized implementation outperforms FIFO across every measured metric in both experiments. Under the balanced workload, the optimized policy beats FIFO by 2-4% on most measurements and reduces the fairness gap between IO and CPU tasks by 37%. Under burst conditions the advantage widens substantially: makespan drops by 8.2%, average wait time by 16.6%, average turnaround by 16.3%, and the fairness gap shrinks by 86%. Worker utilization climbs from 64.0% to 69.4%, meaning the optimized policy keeps an additional half-worker busy on average across the run.

By contrast, FIFO's metrics are nearly identical between balanced and burst workloads, within 1% across all five measurements. This invariance reveals that FIFO is workload-blind: it cannot exploit the visibility that bursts provide because its dispatch logic only ever considers the front of the queue. The optimized policy's four-pass scan, on the other hand, treats a burst as an

opportunity. When ten tasks arrive at once and sit in the queue together, the scan can find the task that best fills the current CPU budget rather than blindly taking whichever task happens to be first. This is why the optimized policy's gains compound under stress: bursts create exactly the conditions where smarter dispatch decisions are possible, and FIFO has no mechanism to capitalize on them. The fairness gap collapse from 458ms to 63ms is the clearest evidence. Under burst conditions the optimized scheduler treats both task types essentially identically, while FIFO maintains the same modest disparity it shows under any workload.

In attempting to overcome the initial FIFO results, research was done through studying the given constraints and possible solutions given in the assignment. It resulted in creating the optimized version’s passes through trial and error. Because the optimized version iterates over the entire queue up to four times, this may create an issue with larger numbers of tasks and a different solution may need to be sought in the future.

Three concurrency bugs surfaced during development. First, the optimized worker initially used notify_all() after every state change, producing a thundering-herd pattern where all eight workers woke on every event and seven went back to sleep having done nothing. Switching to notify_one() reduced lock contention. Second, an early aging implementation incremented skip counts but did not actually protect aged tasks from being passed over again. They simply accumulated higher skip counts until the queue drained at the tail, leaving CPU tasks waiting up to 14 seconds. The fix was a hard-block rule: once any task is aged, the dispatcher refuses to take any other task on that worker until the aged task runs. Third, an attempt to dedicate workers by type (six IO-only, two CPU-only) made performance worse than FIFO because the partition prevented IO workers from helping drain the CPU backlog. Abandoning the partition in favor of a single shared pool with budget-aware preference resolved this.

This study compared a FIFO task dispatcher to a budget-aware dispatcher with aging-based starvation prevention, using both a balanced steady-arrival workload and a stressed burst-arrival workload. The optimized policy outperformed FIFO across every measured metric in both experiments. This pattern suggests that intelligent scheduling is concentrated in the conditions where scheduling decisions matter most. FIFO remains attractive for its simplicity and predictability, but for workloads whose arrival shape cannot be assumed steady in advance, the budget-aware approach is the better default.